Insertion and Deletion of a node using Singly Linked List

1. Insertion of a node in the beginning of a Singly Linked List

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node{
    int data;
    struct Node *next;
};
struct Node *head = NULL;
void insertAtbegining(int x){
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = head;
    head = newNode;
}
void display(){
    struct Node *temp = head;
    while(temp){
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
}
int main(){
    insertAtbegining(10);
    insertAtbegining(20);
    insertAtbegining(30);
    display();
}
```

Output

```
30 -> 20 -> 10 ->

=== Code Execution Successful ===
```

2. Insertion of a node in the ending of a node using Singly Linked List

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
struct Node *head = NULL;
void insertAtending(int x) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = x;
    newNode->next = NULL;
    if (head == NULL) {
        head = newNode;
        return;
    }
    struct Node *temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
void display() {
    struct Node *temp = head;
    if (temp == NULL) {
        printf("List is empty.\n");
        return;
    }
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

Output

```
10 -> 20 -> 30 -> NULL


=== Code Execution Successful ===
```

3. Insertion of a node in the given position of a node using Singly Linked List

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next, *prev;
};
void insertAtPosition(struct Node** head, int data, int pos) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    if (pos == 0) {
        newNode->next = *head;
        newNode->prev = NULL;
        if (*head != NULL) (*head)->prev = newNode;
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    for (int i = 0; i < pos - 1 && temp != NULL; i++) temp = temp->next;
    if (temp == NULL) return;
    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != NULL) temp->next->prev = newNode;
    temp->next = newNode;
}
int main() {
    struct Node* head = NULL;
    insertAtPosition(&head, 10, 0);
    insertAtPosition(&head, 30, 1);
    insertAtPosition(&head, 20, 1);
    struct Node* temp = head;
    while(temp) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
    return 0;
}
```

Output

```
10 <-> 20 <-> 30 <-> NULL


=== Code Execution Successful ===
```

4. Deletion of a node in the beginning using Singly Linked List

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
void deleteFromBeginning(struct Node** head) {
    if (*head == NULL) {
        printf("List is already empty.\n");
        return;
    }
    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
    printf("\n");
}
int main() {
    struct Node* head = (struct Node*)malloc(sizeof(struct Node));
    head->data = 10;
    head->next = (struct Node*)malloc(sizeof(struct Node));
    head->next->data = 20;
    head->next->next = NULL;
    printf("Original ");
    struct Node* curr = head;
    while(curr) {
        printf("%d -> ", curr->data);
        curr = curr->next;
    }
    printf("NULL\n");
    deleteFromBeginning(&head);
    printf("After Deletion: ");
    curr = head;
    while(curr) {
        printf("%d -> ", curr->data);
        curr = curr->next;
    }
    printf("NULL\n");
    return 0;
```

Output

```
Original 10 -> 20 -> NULL

After Deletion: 20 -> NULL


=== Code Execution Successful ===
```

5. Deletion of a node in the ending using Singly Linked List

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
void deleteLast(struct Node** head) {
    if (*head == NULL) return;
    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
    } else {
        struct Node* temp = *head;
        while (temp->next->next != NULL)
            temp = temp->next;

        free(temp->next);
        temp->next = NULL;
    }
}
void insert(struct Node** head, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = NULL;
    if (*head == NULL) { *head = newNode; return; }
    struct Node* temp = *head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
}

void printList(struct Node* head) {
    while (head) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

```

Output

```
Original: 10 -> 20 -> 30 -> NULL
After 1st deletion: 10 -> 20 -> NULL
After 2nd deletion: 10 -> NULL


=== Code Execution Successful ===
```

main.c

```c

    free(temp->next);
    temp->next = NULL;
    }
}
void insert(struct Node** head, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = NULL;
    if (*head == NULL) { *head = newNode; return; }
    struct Node* temp = *head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
}

void printList(struct Node* head) {
    while (head) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;
    insert(&head, 10);
    insert(&head, 20);
    insert(&head, 30);
    printf("Original: ");
    printList(head);
    deleteLast(&head);
    printf("After 1st deletion: ");
    printList(head);
    deleteLast(&head);
    printf("After 2nd deletion: ");
    printList(head);
    return 0;
}
```

Output

```
Original: 10 -> 20 -> 30 -> NULL
After 1st deletion: 10 -> 20 -> NULL
After 2nd deletion: 10 -> NULL


=== Code Execution Successful ===
```

6. Deletion of a node in the given position using Singly Linked list

Programiz
C Online Compiler

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
void deleteAtPosition(struct Node** head, int position) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    if (position == 0) {
        *head = temp->next;
        free(temp);
        return;
    }
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Position out of range\n");
        return;
    }
    struct Node* nextNode = temp->next->next;
    free(temp->next);
    temp->next = nextNode;
}
void insert(struct Node** head, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->next = *head;
    *head = newNode;
}
void printList(struct Node* head) {
    while (head) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}
```

Output

Clear

```
Original: 10 -> 20 -> 30 -> 40 -> NULL
After deleting position 2: 10 -> 20 -> 40 -> NULL


=== Code Execution Successful ===
```